

飞腾 LAPACK 线性代数库用户指南

V1.2 (2024-12-30)

版权所有 © 飞腾信息技术有限公司 2022。保留所有权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium和其他飞腾商标均为飞腾信息技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

本文档的内容视为飞腾的保密信息，您应当严格遵守保密义务；未经飞腾事先书面同意，您不得向任何第三方披露本文档内容或提供给任何第三方使用。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，飞腾在现有技术的基础上尽最大努力提供相应的介绍及操作指引，但飞腾在此明确声明对本文档内容的准确性、完整性、适用性、可靠性等不作任何明示或暗示的保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

本文档中所有内容，包括但不限于图片、架构设计、页面布局、文字描述，均由飞腾和/或其关联公司依法拥有其知识产权，包括但不限于商标权、专利权、著作权等。非经飞腾和/或其关联公司书面同意，任何人不得擅自使用、修改、复制上述内容。
如有技术问题，可联系 support@phytium.com.cn 获取支持。

飞腾信息技术有限公司

地址：天津市滨海新区海缘路1号信安创业广场5号楼

网址：www.phytium.com.cn

版本历史

文档版本	发布日期	更新说明
V1.0	2024-06-30	首次发布。
V1.1	2024-09-30	新增优化接口
V1.2	2024-12-30	新增优化接口

目 录

1 引言	1
2 安装和使用	2
2.1 环境要求	2
2.1.1 硬件环境	2
2.1.2 软件环境	2
2.2 安装 LAPACK-FT	3
2.3 使用 LAPACK-FT	3
2.3.1 线程控制	3
2.3.2 Fortran 调用函数接口示例	3
2.3.3 C 调用函数接口示例 1	7
2.3.4 C 调用函数接口示例 2	11
3 LAPACK-FT 函数说明	15
3.1 函数说明	15
3.2 数据结构	15
4 函数定义	16
4.1 线性求解	16
4.1.1 ?gesv	16
4.1.2 ?ppsv	17
4.1.3 ?ptsv	18
4.1.4 ?gtsv	20
4.1.5 ?posv	21
4.2 矩阵分解函数	22
4.2.1 ?getrf	22
4.2.2 ?geqrf	23
4.2.3 ?gerqf	24

4.2.4 ?geqlf	25
4.2.5 ?gelqf	27
4.2.6 ?potrf	28
4.2.7 ?pttrf	29
4.2.8 ?gttrf	30
4.2.9 ?ssptrf	31
4.2.10 ?pptrf	32
4.3 回代求解函数	33
4.3.1 ?pttrs	33
4.3.2 ?ptts2	34
4.3.3 ?gttrs	35
4.3.4 ?trtrs	36
4.4 矩阵求逆函数	38
4.4.1 ?getri	38
4.4.2 ?potri	39
4.4.3 ?pptri	40
4.4.4 ?trtri	41
4.5 特征值问题函数	42
4.5.1 ?{he,sy}evd	42
4.5.2 ?{he,sy}ev	44
4.5.3 ?steqr	45
4.6 其他函数	47
4.6.1 ?{un,or}glq	47
4.6.2 ?{un,or}gqr	48
4.6.3 ?{un,or}grq	49
4.6.4 ?{or,un}gql	51
4.6.5 ?{or,un}mlq	52
4.6.6 ?{or,un}mqr	54
4.6.7 ?{or,un}mql	55
4.6.8 ?{or,un}mrq	57
4.6.9 ?{he,sy}trd	59
4.6.10 ?lasr	60
4.6.11 ?{he,sy}trd	62

图 目 录

表 目 录

表 2- 1 软件环境2

表 2- 2 硬件环境 2

表 3- 1 数据类型表15

1 引言

飞腾线性代数库 LAPACK-FT (Linear Algebra PACKage for Phytium) 用于解决复杂的线性代数问题, 包括求解线性方程组、线性最小二乘、特征值和奇异值等问题。LAPACK-FT 基于飞腾处理器体系结构特点和指令集特点, 通过向量化、算法改进等优化方法, 实现了在飞腾平台上的性能提升。

2

安装和使用

本章节介绍了如何在飞腾平台上安装使用 LAPACK-FT。

2.1 环境要求

2.1.1 硬件环境

LAPACK-FT 函数库已经验证的硬件环境如表 2-1 所示。

表 2-1 软件环境

项目	说明
CPU	FT-2000+/64、飞腾腾云 S2500、 FT-2000/4、飞腾腾锐 D2000、S5000C

2.1.2 软件环境

LAPACK-FT 函数库已经验证的软件环境如表 2-2 所示。

表 2-2 硬件环境

项目	版本	下载地址
OS	CentOS8.2、Kylin V10、 Ubuntu19.10、OpenEuler22.03	
GCC	7.3.0、8.3.1、9.3.0、10.3.1、11.3.0	
GNU Binutils	2.32、2.33、2.34、2.35、2.37	
Glibc	2.28、2.23、2.31、2.30、2.34	

2.2 安装 LAPACK-FT

步骤一 获取并解压 LAPACK-FT.zip，解压缩目录下包含 LAPACK-FT 的库文件以及头文件。

步骤二 设置库路径。

生成的库在解压文件夹中的 lib 目录下。

```
export LD_LIBRARY_PATH=$HOME/LAPACK-FT/lib:$LD_LIBRARY_PATH
```

2.3 使用 LAPACK-FT

2.3.1 线程控制

通过环境变量可以控制线程的数量：

```
export OMP_NUM_THREADS=4
```

2.3.2 Fortran 调用函数接口示例

步骤 1 创建 test.f 文件。

步骤 2 test.f 文件中添加以下代码内容。

```
* =====  
* DGELS Example.  
* =====  
* Program computes the least squares solution to the overdetermined linear  
* system  $A \cdot X = B$  with full rank matrix A using QR factorization,  
* where A is the coefficient matrix:  
*  
*   1.44  -7.84  -4.39   4.53  
*  -9.96  -0.28  -3.24   3.83  
*  -7.55   3.24   6.27  -6.64  
*   8.34   8.09   5.28   2.06  
*   7.08   2.52   0.74  -2.47  
*  -5.45  -5.70  -1.19   4.70  
*  
* and B is the right-hand side matrix:  
*  
*   8.58   9.35  
*   8.26  -4.43  
*   8.48  -0.70  
*  -5.28  -0.26  
*   5.72  -7.36
```

```
*      8.93  -2.52
*
* Description.
* =====
*
* The routine solves overdetermined or underdetermined real linear systems
* involving an m-by-n matrix A, or its transpose, using a QR or LQ
* factorization of A. It is assumed that A has full rank.
*
* Several right hand side vectors b and solution vectors x can be handled
* in a single call; they are stored as the columns of the m-by-nrhs right
* hand side matrix B and the n-by-nrhs solution matrix X.
*
* Example Program Results.
* =====
*
* DGELS Example Program Results
*
* Solution
*  -0.45   0.25
*  -0.85  -0.90
*   0.71   0.63
*   0.13   0.14
*
* Residual sum of squares for the solution
* 195.36 107.06
*
* Details of QR factorization
* -17.54  -4.76  -1.96   0.42
*  -0.52  12.40   7.88  -5.84
*  -0.40  -0.14  -5.75   4.11
*   0.44  -0.66  -0.20  -7.78
*   0.37  -0.26  -0.17  -0.15
*  -0.29   0.46   0.41   0.24
*
* =====
*
* .. Parameters ..
* INTEGER          M, N, NRHS
* PARAMETER        ( M = 6, N = 4, NRHS = 2 )
* INTEGER          LDA, LDB
* PARAMETER        ( LDA = M, LDB = M )
* INTEGER          LWMAX
* PARAMETER        ( LWMAX = 100 )
```

```

*
*
* .. Local Scalars ..
*      INTEGER          INFO, LWORK
*
*
* .. Local Arrays ..
*      DOUBLE PRECISION A( LDA, N ), B( LDB, NRHS ), WORK( LWMAX )
*      DATA            A/
*      $  1.44,-9.96,-7.55, 8.34, 7.08,-5.45,
*      $ -7.84,-0.28, 3.24, 8.09, 2.52,-5.70,
*      $ -4.39,-3.24, 6.27, 5.28, 0.74,-1.19,
*      $  4.53, 3.83,-6.64, 2.06,-2.47, 4.70
*      $
*      DATA            B/
*      $  8.58, 8.26, 8.48,-5.28, 5.72, 8.93,
*      $  9.35,-4.43,-0.70,-0.26,-7.36,-2.52
*      $
*
*
* .. External Subroutines ..
*      EXTERNAL          DGELS
*      EXTERNAL          PRINT_MATRIX, PRINT_VECTOR_NORM
*
*
* .. Intrinsic Functions ..
*      INTRINSIC          INT, MIN
*
*
* .. Executable Statements ..
*      WRITE(*,*)'DGELS Example Program Results'
*
*      Query the optimal workspace.
*
*      LWORK = -1
*      CALL DGELS( 'No transpose', M, N, NRHS, A, LDA, B, LDB, WORK,
*      $          LWORK, INFO )
*      LWORK = MIN( LWMAX, INT( WORK(1) ) )
*
*      Solve the equations A*X = B.
*
*      CALL DGELS( 'No transpose', M, N, NRHS, A, LDA, B, LDB, WORK,
*      $          LWORK, INFO )
*
*      Check for the full rank.
*
*      IF( INFO.GT.0 ) THEN
*          WRITE(*,*)'The diagonal element ',INFO,' of the triangular '

```

```

        WRITE(*,*)'factor of A is zero, so that A does not have full '
        WRITE(*,*)'rank; the least squares solution could not be '
        WRITE(*,*)'computed.'
        STOP
    END IF

*
*   Print least squares solution.
*
    CALL PRINT_MATRIX( 'Least squares solution', N, NRHS, B, LDB )

*
*   Print residual sum of squares for the solution
*
    CALL PRINT_VECTOR_NORM(
$      'Residual sum of squares for the solution', M-N, NRHS,
$      B( N+1, 1 ), LDB )

*
*   Print details of QR factorization.
*
    CALL PRINT_MATRIX( 'Details of QR factorization', M, N, A, LDA )
    STOP
    END

*
*   End of DGELS Example.
*
*   ===== *
Auxiliary routine: printing a matrix.
*
    SUBROUTINE PRINT_MATRIX( DESC, M, N, A, LDA )
    CHARACTER*(*)    DESC
    INTEGER          M, N, LDA
    DOUBLE PRECISION A( LDA, * )

*
    INTEGER          I, J

*
    WRITE(*,*)
    WRITE(*,*) DESC
    DO I = 1, M
        WRITE(*,9998) ( A( I, J ), J = 1, N )
    END DO

*
9998 FORMAT( 11(:,1X,F6.2) )
    RETURN
    END

```

```
*  
* Auxiliary routine: printing norms of matrix columns.  
*  
SUBROUTINE PRINT_VECTOR_NORM( DESC, M, N, A, LDA )  
CHARACTER*(*) DESC  
INTEGER M, N, LDA  
DOUBLE PRECISION A( LDA, * )  
*  
DOUBLE PRECISION TEMP  
INTEGER I, J  
*  
WRITE(*,*)  
WRITE(*,*) DESC  
DO J = 1, N  
TEMP = 0.0  
DO I = 1, M  
TEMP = TEMP + A( I, J ) * A( I, J )  
END DO  
WRITE(*,9998,ADVANCE='NO') TEMP  
END DO  
WRITE(*,*)  
*  
9998 FORMAT( 11(:,1X,F6.2) )  
RETURN  
END
```

步骤 3 编译

```
gfortran -o test test.f -L$HOME/LAPACK-FT/lib -llapack -llapackex
```

2.3.3 C 调用函数接口示例 1

说明 利用 c 调用 Fortran 接口。

步骤 1 创建 test.c 文件。

步骤 2 test.c 文件中添加以下代码内容。

```
/*  
DGELS Example.  
=====
```

Program computes the least squares solution to the overdetermined linear system $A \cdot X = B$ with full rank matrix A using QR factorization, where A is the coefficient matrix:

```
1.44 -7.84 -4.39 4.53
-9.96 -0.28 -3.24 3.83
-7.55 3.24 6.27 -6.64
8.34 8.09 5.28 2.06
7.08 2.52 0.74 -2.47
-5.45 -5.70 -1.19 4.70
```

and B is the right-hand side matrix:

```
8.58 9.35
8.26 -4.43
8.48 -0.70
-5.28 -0.26
5.72 -7.36
8.93 -2.52
```

Description.

=====

The routine solves overdetermined or underdetermined real linear systems involving an m -by- n matrix A , or its transpose, using a QR or LQ factorization of A . It is assumed that A has full rank.

Several right hand side vectors b and solution vectors x can be handled in a single call; they are stored as the columns of the m -by- $nrhs$ right hand side matrix B and the n -by- $nrhs$ solution matrix X .

Example Program Results.

=====

DGELS Example Program Results

Solution

```
-0.45 0.25
-0.85 -0.90
0.71 0.63
0.13 0.14
```

Residual sum of squares for the solution

```
195.36 107.06
```

Details of QR factorization

```
-17.54 -4.76 -1.96 0.42
```

```
-0.52 12.40 7.88 -5.84
-0.40 -0.14 -5.75 4.11
0.44 -0.66 -0.20 -7.78
0.37 -0.26 -0.17 -0.15
-0.29 0.46 0.41 0.24
*/
#include <stdlib.h>
#include <stdio.h>

/* DGELS prototype */
extern void dgels_( char* trans, int* m, int* n, int* nrhs, double* a, int* lda,
                  double* b, int* ldb, double* work, int* lwork, int* info );
/* Auxiliary routines prototypes */
extern void print_matrix( char* desc, int m, int n, double* a, int lda );
extern void print_vector_norm( char* desc, int m, int n, double* a, int lda );

/* Parameters */
#define M 6
#define N 4
#define NRHS 2
#define LDA M
#define LDB M

/* Main program */
int main() {
    /* Locals */
    int m = M, n = N, nrhs = NRHS, lda = LDA, ldb = LDB, info, lwork;
    double wkopt;
    double* work;
    /* Local arrays */
    double a[LDA*N] = {
        1.44, -9.96, -7.55, 8.34, 7.08, -5.45,
        -7.84, -0.28, 3.24, 8.09, 2.52, -5.70,
        -4.39, -3.24, 6.27, 5.28, 0.74, -1.19,
        4.53, 3.83, -6.64, 2.06, -2.47, 4.70
    };
    double b[LDB*NRHS] = {
        8.58, 8.26, 8.48, -5.28, 5.72, 8.93,
        9.35, -4.43, -0.70, -0.26, -7.36, -2.52
    };
    /* Executable statements */
    printf( " DGELS Example Program Results\n" );
    /* Query and allocate the optimal workspace */
```



```
lwork = -1;
dgelss_( "No transpose", &m, &n, &nrhs, a, &lda, b, &ldb, &wkopt, &lwork,
        &info );
lwork = (int)wkopt;
work = (double*)malloc( lwork*sizeof(double) );
/* Solve the equations A*X = B */
dgelss_( "No transpose", &m, &n, &nrhs, a, &lda, b, &ldb, work, &lwork,
        &info );
/* Check for the full rank */
if( info > 0 ) {
    printf( "The diagonal element %i of the triangular factor ", info );
    printf( "of A is zero, so that A does not have full rank;\n" );
    printf( "the least squares solution could not be computed.\n" );
    exit( 1 );
}
/* Print least squares solution */
print_matrix( "Least squares solution", n, nrhs, b, ldb );
/* Print residual sum of squares for the solution */
print_vector_norm( "Residual sum of squares for the solution", m-n, nrhs,
\
                    &b[n], ldb );
/* Print details of QR factorization */
print_matrix( "Details of QR factorization", m, n, a, lda );
/* Free workspace */
free( (void*)work );
exit( 0 );
} /* End of DGELS Example */

/* Auxiliary routine: printing a matrix */
void print_matrix( char* desc, int m, int n, double* a, int lda ) {
    int i, j;
    printf( "\n %s\n", desc );
    for( i = 0; i < m; i++ ) {
        for( j = 0; j < n; j++ ) printf( " %6.2f", a[i+j*lda] );
        printf( "\n" );
    }
}

/* Auxiliary routine: printing norms of matrix columns */
void print_vector_norm( char* desc, int m, int n, double* a, int lda ) {
    int i, j;
    double norm;
    printf( "\n %s\n", desc );
```

```
for( j = 0; j < n; j++ ) {  
    norm = 0.0;  
    for( i = 0; i < m; i++ ) norm += a[i+j*lda] * a[i+j*lda];  
    printf( " %6.2f", norm );  
}  
printf( "\n" );  
}
```

步骤 3 编译

```
gcc -o test test.c -L$HOME/LAPACK-FT/lib -llapack -llapackex -lgfortran  
-lm
```

2.3.4 C 调用函数接口示例 2

步骤 1 创建 test.c 文件。

步骤 2 test.c 文件中添加以下代码内容。

```
/*  
LAPACKE_dgels Example.  
=====
```

Program computes the least squares solution to the overdetermined linear system $A \cdot X = B$ with full rank matrix A using QR factorization, where A is the coefficient matrix:

```
1.44 -7.84 -4.39 4.53  
-9.96 -0.28 -3.24 3.83  
-7.55 3.24 6.27 -6.64  
8.34 8.09 5.28 2.06  
7.08 2.52 0.74 -2.47  
-5.45 -5.70 -1.19 4.70
```

and B is the right-hand side matrix:

```
8.58 9.35  
8.26 -4.43  
8.48 -0.70  
-5.28 -0.26  
5.72 -7.36  
8.93 -2.52
```

Description.
=====

The routine solves overdetermined or underdetermined real linear systems involving an m -by- n matrix A , or its transpose, using a QR or LQ factorization of A . It is assumed that A has full rank.

Several right hand side vectors b and solution vectors x can be handled in a single call; they are stored as the columns of the m -by- $nrhs$ right hand side matrix B and the n -by- $nrhs$ solution matrix X .

Example Program Results.

=====

LAPACKE_dgels (row-major, high-level) Example Program Results

Solution

```
-0.45  0.25
-0.85 -0.90
 0.71  0.63
 0.13  0.14
```

Residual sum of squares for the solution

195.36 107.06

Details of QR factorization

```
-17.54 -4.76 -1.96  0.42
 -0.52 12.40  7.88 -5.84
 -0.40 -0.14 -5.75  4.11
  0.44 -0.66 -0.20 -7.78
  0.37 -0.26 -0.17 -0.15
 -0.29  0.46  0.41  0.24
```

*/

```
#include <stdlib.h>
```

```
#include <stdio.h>
```

```
#include "lapacke.h"
```

```
/* Auxiliary routines prototypes */
```

```
extern void print_matrix( char* desc, int m, int n, double* a, int lda );
```

```
extern void print_vector_norm( char* desc, int m, int n, double* a, int lda );
```

```
/* Parameters */
```

```
#define M 6
```

```
#define N 4
```

```
#define NRHS 2
```

```
#define LDA N
```

```
#define LDB NRHS

/* Main program */
int main() {
    /* Locals */
    int m = M, n = N, nrhs = NRHS, lda = LDA, ldb = LDB, info;
    /* Local arrays */
    double a[LDA*M] = {
        1.44, -7.84, -4.39, 4.53,
        -9.96, -0.28, -3.24, 3.83,
        -7.55, 3.24, 6.27, -6.64,
        8.34, 8.09, 5.28, 2.06,
        7.08, 2.52, 0.74, -2.47,
        -5.45, -5.70, -1.19, 4.70
    };
    double b[LDB*M] = {
        8.58, 9.35,
        8.26, -4.43,
        8.48, -0.70,
        -5.28, -0.26,
        5.72, -7.36,
        8.93, -2.52
    };
    /* Executable statements */
    printf( "LAPACKE_dgels (row-major, high-level) Example Program\n");
    printf( "Results\n");
    /* Solve the equations A*X = B */
    info = LAPACKE_dgels( LAPACK_ROW_MAJOR, 'N', m, n, nrhs, a, lda,
                          b, ldb );
    /* Check for the full rank */
    if( info > 0 ) {
        printf( "The diagonal element %i of the triangular factor", info );
        printf( "of A is zero, so that A does not have full rank;\n" );
        printf( "the least squares solution could not be computed.\n" );
        exit( 1 );
    }
    /* Print least squares solution */
    print_matrix( "Least squares solution", n, nrhs, b, ldb );
    /* Print residual sum of squares for the solution */
    print_vector_norm( "Residual sum of squares for the solution", m-n, nrhs,
                      &b[n*ldb], ldb );
    /* Print details of QR factorization */
    print_matrix( "Details of QR factorization", m, n, a, lda );
}
```

```
        exit( 0 );
    } /* End of LAPACK_E_dgels Example */

/* Auxiliary routine: printing a matrix */
void print_matrix( char* desc, int m, int n, double* a, int lda ) {
    int i, j;
    printf( "\n %s\n", desc );
    for( i = 0; i < m; i++ ) {
        for( j = 0; j < n; j++ ) printf( " %6.2f", a[i*lda+j] );
        printf( "\n" );
    }
}

/* Auxiliary routine: printing norms of matrix columns */
void print_vector_norm( char* desc, int m, int n, double* a, int lda ) {
    int i, j;
    double norm;
    printf( "\n %s\n", desc );
    for( j = 0; j < n; j++ ) {
        norm = 0.0;
        for( i = 0; i < m; i++ ) norm += a[i*lda+j] * a[i*lda+j];
        printf( " %6.2f", norm );
    }
    printf( "\n" );
}
```

步骤 3 编译

```
gcc -o test test.c -I$HOME/LAPACK-FT/include -L$HOME/LAPACK-FT/lib
-llapacke -llapack -llapackex -lgfortran -lm
```

3 LAPACK-FT 函数说明

3.1 函数说明

- 1、线性求解：用于解决形如 $A \cdot X = B$ 的线性方程组，其中 A 是一个已知矩阵， X 和 B 是待求解的矩阵。
- 2、矩阵分解函数：用于将给定的矩阵进行分解，常见的分解包括 LU 分解、QR 分解、Cholesky 分解等。
- 3、回代求解函数：用于解决上三角或下三角线性方程组，通常称为回代（Back Substitution）。
- 4、矩阵求逆函数：用于计算给定矩阵的逆矩阵，即如果 A 是一个矩阵，这些函数可以计算出矩阵 A 的逆矩阵。
- 5、特征值问题函数：用于计算矩阵的特征值和特征向量，例如，非对称特征值问题和对称特征值问题。

3.2 数据结构

<数据类型>说明，如表 3-1 所示

表 3-1 数据类型表

<数据类型>	说明
s	单精度浮点类型
d	双精度浮点类型
c	单精度复数类型
z	双精度复数类型

4 函数定义

4.1 线性求解

4.1.1 ?gesv

接口说明: 求解线性方程 $A * X = B$, 其中 A 是一个 $N \times N$ 矩阵, X 和 B 是 $N \times NRHS$ 矩阵。使用 LU 分解的方法, 配合部分主元选取和行置换, 将矩阵 A 分解为三个部分: P 、 L 和 U , 其中 P 是一个置换矩阵, L 是单位下三角矩阵, U 是上三角矩阵。然后使用 A 的分解形式来求解方程组 $A * X = B$ 。

接口定义

Fortran 接口:

```
subroutine sgesv (n, nrhs, a, lda, ipiv, b, ldb, info);  
subroutine dgesv (n, nrhs, a, lda, ipiv, b, ldb, info);  
subroutine cgesv (n, nrhs, a, lda, ipiv, b, ldb, info);  
subroutine zgesv (n, nrhs, a, lda, ipiv, b, ldb, info);
```

C 接口:

```
void sgesv_( int* n, int* nrhs, float* a, int* lda, int* ipiv, float* b, int* ldb, int*  
info );  
void dgesv_( int* n, int* nrhs, double* a, int* lda, int* ipiv, double* b, int* ldb,  
int* info );  
void cgesv_( int* n, int* nrhs, float _Complex* a, int* lda, int* ipiv, float  
_Complex* b, int* ldb, int* info );
```

```
void zgesv_( int* n, int* nrhs, double _Complex* a, int* lda, int* ipiv, double
_Complex* b, int* ldb, int* info );
```

参数说明:

参数名	类型	描述	输入/输出
n	整型数	矩阵 A 的阶数 。	输入
nrhs	整型数	矩阵 B 的列数, nrhs≥0。	输入
a	对应类型的数组	矩阵维度为(lda,n)。 输入: n*n 的系数矩阵 A。 输出: 储存了 A=P*L*U 中的 L 和 U。	输入/输出
lda	整型数	A 的 leading dimension 大小 , 要求 lda≥max(1, n)。	输入
ipiv	整形数组	数组维度为 n。 存放了置换矩阵 P 的主元索引, 即矩阵第 i 行与 第 ipiv(i)行交换。	输出
b	对应类型的数组	矩阵维度为(ldb,nrhs)。 输入: 点击放大的右端矩阵 B。 输出: 在 info=0 的情况下, 为 n×nrhs 的解矩 阵 X。	输入/输出
ldb	整型数	B 的 leading dimension 大小 , 要求 ldb≥max(1, n)	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i, 则第 i 个参数具有非法 值 > 0: 如果 INFO = i, 则 U(i,i) 恰好为零。因 式分解已完成, 但因子 U 恰好是奇异的, 因此 无法计算解。	输出

4.1.2 ?ppsv

接口说明: 计算具有对称 (厄米特) 正定填充系数矩阵 A 和多个右侧的线性方程组的解。

接口定义

Fotran 接口:


```
subroutine sppsv (uplo, n, nrhs, ap, b, ldb, info);  
subroutine dppsv (uplo, n, nrhs, ap, b, ldb, info);  
subroutine cppsv (uplo, n, nrhs, ap, b, ldb, info);  
subroutine zppsv (uplo, n, nrhs, ap, b, ldb, info);
```

C 接口:

```
void dppsv_(const char *uplo, const int *n, const int *nrhs, double *ap,  
double *b, const int *ldb, int *info);  
void sppsv_(const char *uplo, const int *n, const int *nrhs, float *ap, float *b,  
const int *ldb, int *info);  
void cppsv_(const char *uplo, const int *n, const int *nrhs, float _Complex  
*ap, float _Complex *b, const int *ldb, int *info);  
void zppsv_(const char *uplo, const int *n, const int *nrhs, double _Complex  
*ap, double _Complex *b, const int *ldb, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的阶, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数, $nrhs \geq 0$	输入
ap	对应类型数组	包含打包存储中矩阵 A 的上三角部分或下三角部分 (由 uplo 指定)。	输入/输出
b	对应类型数组	调用前为右侧矩阵 B。调用后为解矩阵 X。	输入/输出
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i, 则第 i 个参数具有非法值。 > 0: 如果 INFO = i, 则 A 的 i 阶主次数不为正, 因此 无法完成因式分解, 并且尚未计算解。	输出

4.1.3 ?ptsv

接口说明: 计算复杂的线性方程组 $A \cdot X = B$ 的解, 其中 A 是 $N \times N$ 埃尔米特正定三对角矩阵, X 和 B 是 $N \times NRHS$ 矩阵。

接口定义

Fortran 接口:

```
subroutine sptsv (n, nrhs, d, e, b, ldb, info);  
subroutine dptsv (n, nrhs, d, e, b, ldb, info);  
subroutine cptsv (n, nrhs, d, e, b, ldb, info);  
subroutine zptsv (n, nrhs, d, e, b, ldb, info);
```

C 接口:

```
void sptsv_(const int *n, const int *nrhs, float *d, float *e, float *b, const int  
*ldb, int *info);  
void dptsv_(const int *n, const int *nrhs, double *d, double *e, double *b,  
const int *ldb, int *info);  
void cptsv_(const int *n, const int *nrhs, float *d, float _Complex *e, float  
_Complex *b, const int *ldb, int *info);  
void zptsv_(const int *n, const int *nrhs, double *d, double _Complex *e,  
double _Complex *b, const int *ldb, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
n	整数型	矩阵 A 的维数, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数, $nrhs \geq 0$ 。	输入
d	在 sptsv/cptsv 中为单精度浮点型数组。 在 dptsv/zptsv 中为双精度浮点型数组。	输入: 三对角矩阵 A 的对角元素, 维度为 n。 输出: 分解后的对角元素 d。	输入/输出
e	对应类型数组	输入: 三对角矩阵 A 的次对角元素, 维度为 $n-1$ 。 输出: 分解后矩阵 A 的次对角元素。	输入/输出
b	对应类型的数组	输入: 右边矩阵, 维度为(ldb, nrhs)。 输出: 结果 X, 维度为(ldb, nrhs)。	输入/输出
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入

参数名	类型	描述	输入/输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i, 则第 i 个参数具有非法值。 > 0: 如果 INFO = i, 则 i 阶的主次数不是正数, 并且尚未计算解。除非 i = N, 否则分解尚未完成。	输出

4.1.4 ?gtsv

接口说明：计算具有三对角系数矩阵 A 和多个右侧的线性方程组的解。

接口定义

Fortran 接口：

```
subroutine sgtsv (n, nrhs, dl, d, du, b, ldb, info);
subroutine dgtsv (n, nrhs, dl, d, du, b, ldb, info);
subroutine cgtsv (n, nrhs, dl, d, du, b, ldb, info);
subroutine zgtsv (n, nrhs, dl, d, du, b, ldb, info);
```

C 接口：

```
void sgtsv(const int *n, const int *nrhs, float *dl, float *d, float *du, float *b,
const int *ldb, int *info);
void dgtsv(const int *n, const int *nrhs, double *dl, double *d, double *du,
double *b, const int *ldb, int *info);
void cgtsv(const int *n, const int *nrhs, float _Complex *dl, float _Complex *d,
float _Complex *du, float _Complex *b, const int *ldb, int *info);
void zgtsv(const int *n, const int *nrhs, double _Complex *dl, double
_Complex *d, double _Complex *du, double _Complex *b, const int *ldb, int
*info);
```

参数说明：

参数名	类型	描述	输入/输出
n	整数型	矩阵 A 的维数， $N \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数， $nrhs \geq 0$ 。	输入
dl	对应类型数组	输入时，为三对角矩阵 A 的次对角元素，维度为 $n-1$ 。 输出时，为 U 的第二超对角元素。	输入/输出

参数名	类型	描述	输入/输出
d	对应类型数组	输入时, 为三对角矩阵 A 的超对角元素, 维度为 $n-1$ 。 输出时, 为 U 的第一超对角元素。	输入/输出
du	对应类型数组	输入时, 为三对角矩阵 A 的超对角元素, 维度为 $N-1$ 。 输出时, 为 U 的第一超对角元素。	输入/输出
b	对应类型数组	右边矩阵, 维度为 $(ldb, nrhs)$ 。	输入
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。 > 0: 如果 $INFO = i$, 则 $U(i,i)$ 恰好为零, 并且尚未计算解。除非 $i = N$, 否则分解尚未完成。	输出

4.1.5 ?posv

接口说明: 计算具有对称或埃尔米特正定系数矩阵 A 和多个右侧的线性方程组的解。

接口定义

Fortran 接口:

```
subroutine sposv(uplo, n, nrhs, a, lda, b, ldb, info);  
subroutine dposv(uplo, n, nrhs, a, lda, b, ldb, info);  
subroutine cposv(uplo, n, nrhs, a, lda, b, ldb, info);  
subroutine zposv(uplo, n, nrhs, a, lda, b, ldb, info);
```

C 接口:

```
void dposv_(const char *uplo, const int *n, const int *nrhs, double *a, const  
int *lda, double *b, const int *ldb, int *info);  
void sposv_(const char *uplo, const int *n, const int *nrhs, float *a, const int  
*lda, float *b, const int *ldb, int *info);  
void cposv_(const char *uplo, const int *n, const int *nrhs, float _Complex *a,  
const int *lda, float _Complex *b, const int *ldb, int *info);  
void zposv_(const char *uplo, const int *n, const int *nrhs, double _Complex  
*a, const int *lda, double _Complex *b, const int *ldb, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的阶, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数, $nrhs \geq 0$	输入
a	对应类型数组	调用前: 包含打包存储中矩阵 A 的上三角部分或下三角部分 (由 uplo 指定)。 调用后: 保存 A 的 Cholesky 分解结果。	输入/输出
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
b	对应类型数组	调用前: 右侧矩阵 B。 调用后: 解矩阵 X。	输入/输出
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。 > 0: 如果 $INFO = i$, 则 A 的 i 阶主次数不为正, 因此无法完成因式分解, 并且尚未计算解。	输出

4.2 矩阵分解函数

4.2.1 ?getrf

接口说明: 计算一般 $m \times n$ 矩阵的 LU 分解。

接口定义

Fortran 接口:

```
subroutine sgetrf(m, n, a, lda, ipiv, info);  
subroutine dgetrf(m, n, a, lda, ipiv, info);  
subroutine cgetrf(m, n, a, lda, ipiv, info);  
subroutine zgetrf(m, n, a, lda, ipiv, info)
```

C 接口:

```
void sgetrf_(const int *m, const int *n, float *a, const int *lda, int *ipiv, int *info);  
void dgetrf_(const int *m, const int *n, double *a, const int *lda, int *ipiv, int *info);  
void cgetrf_(const int *m, const int *n, float _Complex *a, const int *lda, int
```

```
*ipiv, int *info);  
void zgetrf_(const int *m, const int *n, double _Complex *a, const int *lda, int  
*ipiv, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	对应类型数组	输入: 矩阵 A。 输出: 保存分解结果 L 和 U	输入/输出
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
ipiv	整型数组	大小为 $\max(1, \min(m, n))$ 的数组。包含主元素索引; 对于 $1 \leq i \leq \min(m, n)$, 第 i 行与第 ipiv(i) 行进行了交换。	输入/输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。 > 0: 如果 $INFO = i$, 则 $U(i,i)$ 恰好为零。因式分解已经完成, 但是因子 U 恰好是奇异的, 如果用它来求解方程组, 就会出现被零除的情况。	输出

4.2.2 ?geqrf

接口说明: 计算一般 $m \times n$ 矩阵的 QR 分解。

接口定义**Fortran 接口:**

```
subroutine sgeqrf(m, n, a, lda, tau, work, lwork, info);  
subroutine dgeqrf(m, n, a, lda, tau, work, lwork, info);  
subroutine cgeqrf(m, n, a, lda, tau, work, lwork, info);  
subroutine zgeqrf(m, n, a, lda, tau, work, lwork, info);
```

C 接口:

```
void dgeqrf_(const int *m, const int *n, double *a, const int *lda, double *tau,  
double *work, const int *lwork, int *info);  
void sgeqrf_(const int *m, const int *n, float *a, const int *lda, float *tau, float
```

```

*work, const int *lwork, int *info);
void cgeqrf_(const int *m, const int *n, float _Complex *a, const int *lda, float
_Complex *tau, float _Complex *work, const int *lwork, int *info);
void zgeqrf_(const int *m, const int *n, double _Complex *a, const int *lda,
double _Complex *tau, double _Complex *work, const int *lwork, int *info);

```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	对应类型数组	调用前保存待分解的矩阵 A。 调用后：对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R，对角线以下元素和 tau 共同表示正交矩阵 Q。	输入/输出
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	数组的大小至少为 $\max(1, \min(m, n))$ 。包含了矩阵 Q 的分解中用于定义初等反射变换的标量值。	输出
work	对应类型数组	临时存储空间，使用 $lwork=-1$ 调用后 work[0] 为最优的 lwork 值。	输出
lwork	整型数	work 数组的长度。 $lwork=-1$ 时查询最优 work 大小，结果保存在 work[0] 中，否则要求 $lwork \geq n$ 。	输入
info	整型数	执行结果。 = 0: 成功退出。 < 0: 如果 $INFO = -i$ ，则第 i 个参数具有非法值。	输出

4.2.3 ?gerqf

接口说明: 计算一般 $m \times n$ 矩阵的 RQ 分解。

接口定义**Fortran 接口:**

```

subroutine sgerqf (m, n, a, lda, tau, work, lwork, info);
subroutine dgerqf (m, n, a, lda, tau, work, lwork, info);
subroutine cgerqf (m, n, a, lda, tau, work, lwork, info);
subroutine zgerqf (m, n, a, lda, tau, work, lwork, info);

```

C 接口:

```
void sgerqf_(const int *M, const int *N, float *A, const int *LDA, float *TAU,
float *WORK, const int *LWORK, int *INFO);
void dgerqf_(const int *M, const int *N, double *A, const int *LDA, double
*TAU, double *WORK, const int *LWORK, int *INFO);
void cgerqf_(const int *M, const int *N, float _Complex *A, const int *LDA,
float _Complex *TAU, float _Complex *WORK, const int *LWORK, int
*INFO);
void zgerqf_(const int *M, const int *N, double _Complex *A, const int *LDA,
double _Complex *TAU, double _Complex *WORK, const int *LWORK, int
*INFO);
```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	对应类型数组	调用前: 保存待分解的矩阵 A。 调用后: 对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R(当 $m \geq n$ 时 R 为上三角矩阵), 对角线以下元素和 tau 共同表示正交矩阵 Q。	输入/输出
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	大小至少为 $\max(1, \min(m, n))$ 。包含矩阵 Q 的初等反射变换的标量因子。	输出
work	对应类型数组	临时存储空间, 使用 $lwork=-1$ 调用后 $work[0]$ 为最优的 $lwork$ 值。	输出
lwork	整型数	work 数组的长度。 $lwork=-1$ 时查询最优 work 大小, 结果保存在 $work[0]$ 中, 否则要求 $lwork \geq n$ 。	输入
info	整型数	执行结果。 = 0: 成功退出。 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。	输出

4.2.4 ?geqlf

接口说明: 计算一般 $m \times n$ 矩阵的 QL 分解。

接口定义

Fortran 接口:

```
subroutine sgeqlf (m, n, a, lda, tau, work, lwork, info);
```

```
subroutine dgeqlf (m, n, a, lda, tau, work, lwork, info);
```

```
subroutine cgeqlf (m, n, a, lda, tau, work, lwork, info);
```

```
subroutine zgeqlf (m, n, a, lda, tau, work, lwork, info);
```

C 接口:

```
void sgeqlf_(const int *M, const int *N, float *A, const int *LDA, float *TAU,  
float *WORK, const int *LWORK, int *INFO);
```

```
void dgeqlf_(const int *M, const int *N, double *A, const int *LDA, double  
*TAU, double *WORK, const int *LWORK, int *INFO);
```

```
void cgeqlf_(const int *M, const int *N, float _Complex *A, const int *LDA,  
float _Complex *TAU, float _Complex *WORK, const int *LWORK, int  
*INFO);
```

```
void zgeqlf_(const int *M, const int *N, double _Complex *A, const int *LDA,  
double _Complex *TAU, double _Complex *WORK, const int *LWORK, int  
*INFO);
```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	对应类型数组	调用前: 保存待分解的矩阵 A。 调用后: 调用后对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R (当 $m \geq n$ 时 L 为下三角矩阵), 对角线以下元素和 tau 共同表示正交矩阵 Q。	输入/输出
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	大小至少为 $\max(1, \min(m, n))$ 。包含矩阵 Q 的初等反射变换的标量因子。	输出
work	对应类型数组	临时存储空间, 使用 $lwork=-1$ 调用后 work[0] 为最优的 lwork 值。	输出
lwork	整型数	work 数组的长度。 $lwork=-1$ 时查询最优 work 大小, 结果保存在 work[0] 中, 否则要求 $lwork \geq n$ 。	输入
info	整型数	执行结果。 = 0: 成功退出。 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。	输出

4.2.5 ?gelqf

接口说明：计算一般 $m \times n$ 矩阵的 LQ 分解。

接口定义

Fortran 接口：

```
subroutine sgelqf (m, n, a, lda, tau, work, lwork, info);
```

```
subroutine dgelqf (m, n, a, lda, tau, work, lwork, info);
```

```
subroutine cgelqf (m, n, a, lda, tau, work, lwork, info);
```

```
subroutine zgelqf (m, n, a, lda, tau, work, lwork, info);
```

C 接口：

```
void sgelqf_(const int *M, const int *N, float *A, const int *LDA, float *TAU,  
float *WORK, const int *LWORK, int *INFO);
```

```
void dgelqf_(const int *M, const int *N, double *A, const int *LDA, double  
*TAU, double *WORK, const int *LWORK, int *INFO);
```

```
void cgelqf_(const int *M, const int *N, float _Complex *A, const int *LDA,  
float _Complex *TAU, float _Complex *WORK, const int *LWORK, int  
*INFO);
```

```
void zgelqf_(const int *M, const int *N, double _Complex *A, const int *LDA,  
double _Complex *TAU, double _Complex *WORK, const int *LWORK, int  
*INFO);
```

参数说明：

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	对应类型数组	调用前：保存待分解的矩阵 A。 调用后：调用后对角线以及对角线以上的部分保存大小为 $\min(m,n) \times n$ 的矩阵 R (当 $m \geq n$ 时 L 为下三角矩阵)，对角线以下元素和 tau 共同表示正交矩阵 Q。	输入/输出
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	大小至少为 $\max(1, \min(m, n))$ 。包含矩阵 Q 的初等反射变换的标量因子。	输出
work	对应类型数组	临时存储空间，使用 $lwork = -1$ 调用后 work[0] 为最优的 lwork 值。	输出

参数名	类型	描述	输入/输出
lwork	整型数	work 数组的长度。lwork=-1 时查询最优 work 大小，结果保存在 work[0] 中，否则要求 $lwork \geq n$ 。	输入
info	整型数	执行结果。 = 0: 成功退出。 < 0: 如果 $INFO = -i$ ，则第 i 个参数具有非法值。	输出

4.2.6 ?potrf

接口说明：计算对称（埃尔米特）正定矩阵的 Cholesky 分解。

接口定义

Fortran 接口：

```
subroutine cpotrf (uplo, n, a, lda, info);
subroutine dpotrf (uplo, n, a, lda, info);
subroutine spotrf (uplo, n, a, lda, info);
subroutine zpotrf (uplo, n, a, lda, info);
```

C 接口：

```
void spotrf_(const char *uplo, const int *n, float *a, const int *lda, int *info);
void dpotrf_(const char *uplo, const int *n, double *a, const int *lda, int *info);
void cpotrf_(const char *uplo, const int *n, float _Complex *a, const int *lda,
int *info);
void zpotrf_(const char *uplo, const int *n, double _Complex *a, const int
*lda, int *info);
```

参数说明：

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的阶数。	输入
a	对应类型数组	大小至少为 $\max(1, lda \cdot n)$ 。包含矩阵 A 的上三角部分或下三角部分（参见 uplo）。	输入/输出
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入

参数名	类型	描述	输入/输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i, 则第 i 个参数具有非法值 > 0: 如果 INFO = i, 则 i 阶的主次数非正, 无法完成分解。	输出

4.2.7 ?pttrf

接口说明: 计算对称 (埃尔米特) 正定三对角矩阵的因子分解。

接口定义

Fortran 接口:

```
subroutine cpttrf(n, d, e, info);
```

```
subroutine dpttrf(n, d, e, info);
```

```
subroutine sptrf(n, d, e, info);
```

```
subroutine zpttrf(n, d, e, info);
```

C 接口:

```
void sgtrf(const int *N, float *DL, float *D, float *DU, float *DU2, int *IPIV, int *INFO);
```

```
void dgtrf(const int *N, double *DL, double *D, double *DU, double *DU2, int *IPIV, int *INFO);
```

```
void cgtrf(const int *N, float _Complex *DL, float _Complex *D, float _Complex *DU, float _Complex *DU2, int *IPIV, int *INFO);
```

```
void zgtrf(const int *N, double _Complex *DL, double _Complex *D, double _Complex *DU, double _Complex *DU2, int *IPIV, int *INFO);
```

参数说明:

参数名	类型	描述	输入/输出
n	整型数	矩阵 a 的阶数。 $n \geq 0$ 。	输入
d	在 sptrf/cpttrf 中为单精度浮点型数组。 在 dpttrf/zpttrf 中为双精度浮点型数组。	输入: 为三对角矩阵 a 的对角元素, 维度为 n。 输出: 为分解后的对角元素 d。	输入/输出

参数名	类型	描述	输入/输出
e	对应类型数组	输入：为三对角矩阵 a 的次对角元素，维度为 n-1。 输出：为分解后矩阵 a 的次对角元素。	输入/输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -k, 则第 k 个参数具有非法值 > 0: 如果 INFO = k, 则 k 阶主次要非正；如果 k < N, 则无法完成分解，如果 k = N, 则完成分解，但 D(N) <= 0。	输出

4.2.8 ?gttrf

接口说明：计算三对角矩阵的 LU 分解。

接口定义

Fortran 接口：

```
subroutine cgtrf (n, dl, d, du, du2, ipiv, info);  
subroutine dgtrf (n, dl, d, du, du2, ipiv, info);  
subroutine sgtrf (n, dl, d, du, du2, ipiv, info);  
subroutine zgtrf (n, dl, d, du, du2, ipiv, info);
```

C 接口：

```
void sptrf_(const int *N, float *D, float *E, int *INFO);  
void dptrf_(const int *N, double *D, double *E, int *INFO);  
void cptrf_(const int *N, float *D, float _Complex *E, int *INFO);  
void zptrf_(const int *N, double *D, double _Complex *E, int *INFO);
```

参数说明：

参数名	类型	描述	输入/输出
n	整型数	矩阵 a 的维数。n ≥ 0。	输入
dl	对应类型数组	输入：三对角矩阵 A 的次对角元素，维度为 N-1。 输出：L 矩阵。	输入/输出
d	对应类型数组	输入：为三对角矩阵 A 的对角元素，维度为 N。 输出：U 的对角元素。	输入/输出
du	对应类型数组	输入：三对角矩阵 A 的超对角元素，维度为 n-1。 输出：U 的第一超对角元素。	输入/输出

参数名	类型	描述	输入/输出
du2	对应类型数组	维度为 $(n-2)$ 。在退出时, du2 包含 U 的第二超对角线的 $(n-2)$ 个元素。	输出
ipiv	整型数	转置数组。	输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -k$, 则第 k 个参数具有非法值。 > 0: 如果 $INFO = k$, 则 $U(k,k)$ 恰好为零。因式分解已经完成, 但是因子 U 恰好是奇异的, 如果用它来求解方程组, 就会出现被零除的情况。	输出

4.2.9 ?ssptrf

接口说明: 使用打包存储计算对称矩阵的 Bunch-Kaufman 分解。

接口定义

Fortran 接口:

```
subroutine ssptrf (uplo, n, ap, ipiv, info);
```

```
subroutine dsptrf (uplo, n, ap, ipiv, info);
```

```
subroutine csptrf (uplo, n, ap, ipiv, info);
```

```
subroutine zsptrf (uplo, n, ap, ipiv, info);
```

C 接口:

```
void ssptrf_(const char *uplo, const int *n, float *ap, int *ipiv, int *info);
```

```
void dsptrf_(const char *uplo, const int *n, double *ap, int *ipiv, int *info);
```

```
void csptrf_(const char *uplo, const int *n, float _Complex *ap, int *ipiv, int *info);
```

```
void zsptrf_(const char *uplo, const int *n, double _Complex *ap, int *ipiv, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
uplo	字符数	'U': 保存 A 的上三角矩阵。 'L': 保存 A 的下三角矩阵。	输入
n	整型数	矩阵 a 的阶数。 $n \geq 0$ 。	输入
ap	对应类型数组	调用前: 为压缩对称矩阵 A, 大小为 $N*(N+1)/2$ 。 调用后: 块对角矩阵 D, 和用于获得因子 U 或 L 的乘法器, 存储为上或下三角矩阵覆盖 A。	输入/输出

参数名	类型	描述	输入/输出
ipiv	整数型数组	大小为 $\max(1,n)$ 。 如果 $UPLO = 'U'$ 且 $IPIV(k) = IPIV(k-1) < 0$, 则行和列 $k-1$ 和 $-IPIV(k)$ 互换, 并且 $D(k-1:k,k-1:k)$ 是一个 2×2 对角线块。 如果 $UPLO = 'L'$ 且 $IPIV(k) = IPIV(k+1) < 0$, 则行和列 $k+1$ 和 $-IPIV(k)$ 互换, 并且 $D(k:k+1,k:k+1)$ 是一个 2×2 对角线块。	输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值 > 0: 如果 $INFO = i$, 则 $D(i,i)$ 恰好为零。因式分解已经完成, 但是块对角矩阵 D 恰好是奇异的, 如果用它来求解方程组, 就会出现被零除的情况。	输出

4.2.10 ?pptrf

接口说明：使用打包存储计算对称（ 埃尔米特 ）正定矩阵的 Cholesky 分解。

接口定义

Fotran 接口：

```
subroutine cpptrf (uplo, n, ap, info);
subroutine dpptrf (uplo, n, ap, info);
subroutine spptrf (uplo, n, ap, info);
subroutine zpptrf (uplo, n, ap, info);
```

C 接口：

```
void spptrf_(const char *uplo, const int *n, float *ap, int *info);
void dpptrf_(const char *uplo, const int *n, double *ap, int *info);
void cpptrf_(const char *uplo, const int *n, float _Complex *ap, int *info);
void zpptrf_(const char *uplo, const int *n, double _Complex *ap, int *info);
```

参数说明：

参数名	类型	描述	输入/输出
uplo	字符数	'U': 保存 A 的上三角矩阵。 'L': 保存 A 的下三角矩阵。	输入
n	整型数	矩阵 a 的阶数。 $n \geq 0$ 。	输入

参数名	类型	描述	输入/输出
ap	对应类型数组	调用前: 矩阵 A 的上/下三角部分, 大小为 $N*(N+1)/2$ 。 调用后: 三角因子 U 或 L, 存储为上或下三角矩阵覆盖 A。	输入/输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。 > 0: 如果 $INFO = i$, 则 i 阶的主次数不是正定的, 无法完成因式分解。	输出

4.3 回代求解函数

4.3.1 ?pttrs

接口说明: 通过?pttrf 计算的分解, 计算一个具有对称 (埃尔米特) 正定三对角系数矩阵的线性方程组。

接口定义

```
subroutine spttrs (n, nrhs, d, e, b, ldb, info);
subroutine dpttrs (n, nrhs, d, e, b, ldb, info);
subroutine cpttrs (uplo, n, nrhs, d, e, b, ldb, info);
subroutine zpttrs (uplo, n, nrhs, d, e, b, ldb, info);
```

Fortran 接口:

C 接口:

```
void spttrs_(const int *N, const int *NRHS, float *D, float *E, float *B, const
int *LDB, int *INFO);
void dpttrs_(const int *N, const int *NRHS, double *D, double *E, double *B,
const int *LDB, int *INFO);
void cpttrs_(const char *UPLO, const int *N, const int *NRHS, float *D, float
_Complex *E, float _Complex *B, const int *LDB, int *INFO);
void zpttrs_(const char *UPLO, const int *N, const int *NRHS, double *D,
double _Complex *E, double _Complex *B, const int *LDB, int *INFO);
```

参数说明:

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
Uplo (复数独有)	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 a 的维数, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 b 的列数, $nrhs \geq 0$ 。	输入
d	在 spttrs/cpttrs 中为单精度浮点型数组。 在 dpttrs/zpttrs 中为双精度浮点型数组。	维度 (n)。包含由 ?pttrf 计算的因式分解的对角矩阵 D 的对角元素。	输入
e	对应类型数组	维度 (n-1)。三对角矩阵 a 的次对角元素。	输入
b	对应类型数组	输入: 右端项矩阵, 维度为(ldb, nrhs)。 输出: 解 X, 维度为(ldb, nrhs)。	输入/输出
ldb	整型数	b 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入
info	整型数	执行结果。 = 0: 成功退出。 < 0: 如果 INFO = -i, 则第 i 个参数具有非法值。	输出

4.3.2 ?ptts2

接口说明: 通过?pttrf 计算的分解, 求解三对角方程 $AX=B$ 。

接口定义

Fortran 接口:

```
subroutine sptts2 (n, nrhs, d, e, b, ldb);
subroutine dptts2 (n, nrhs, d, e, b, ldb);
subroutine cptts2 (iuplo, n, nrhs, d, e, b, ldb);
subroutine zptts2 (iuplo, n, nrhs, d, e, b, ldb);
```

C 接口:

```
void sptts2_(const int *N, const int *NRHS, float *D, float *E, float *B, const int *LDB);
void dptts2_(const int *N, const int *NRHS, double *D, double *E, double *B, const int *LDB);
void cptts2_(const int *IUPLO, const int *N, const int *NRHS, float *D, float _Complex *E, float _Complex *B, const int *LDB);
void zptts2_(const int *IUPLO, const int *N, const int *NRHS, double *D,
```

```
double _Complex *E, double _Complex *B, const int *LDB);
```

参数说明:

参数名	类型	描述	输入/输出
iuplo (复数独有)	整型数	= 1: $A = U^{**H} * D * U$, E 是 U 的上对角线。 = 0: $A = L * D * L^{**H}$, E 是 L 的下对角线。	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数, $nrhs \geq 0$ 。	输入
d	在 sptts2/cptts2 中 为单精度浮点型数组。 在 dptts2/zptts2 中 为双精度浮点型数组。	维度 (n)。三对角矩阵 A 的对角元素	输入
e	对应类型数组	维度 (n-1)。三对角矩阵 a 的次对角元素。	输入
b	对应类型数组	输入: 右端项矩阵, 维度为(ldb, nrhs)。 输出: 解 X, 维度为(ldb, nrhs)。	输入/输出
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入

4.3.3 ?gttrs

接口说明: 使用 ?gttrf 计算的 LU 分解求解具有三对角系数矩阵的线性方程组。

接口定义

Fortran 接口:

```
subroutine cgtrrs (trans, n, nrhs, dl, d, du, du2, ipiv, b, ldb, info);
```

```
subroutine dgtrrs (trans, n, nrhs, dl, d, du, du2, ipiv, b, ldb, info);
```

```
subroutine sgtrrs (trans, n, nrhs, dl, d, du, du2, ipiv, b, ldb, info);
```

```
subroutine zgtrrs (trans, n, nrhs, dl, d, du, du2, ipiv, b, ldb, info);
```

C 接口:

```
void sgtrrs(const char *TRANS, const int *N, const int *NRHS, float *DL,  
float *D, float *DU, float *DU2, int *IPIV, float *B, const int *LDB, int *INFO);
```

```
void dgtrrs(const char *TRANS, const int *N, const int *NRHS, double *DL,  
double *D, double *DU, double *DU2, int *IPIV, double *B, const int *LDB, int  
*INFO);
```

```
void cgtrrs(const char *TRANS, const int *N, const int *NRHS, float
```

```

_Complex *DL, float _Complex *D, float _Complex *DU, float _Complex
*DU2, int *IPIV, float _Complex *B, const int *LDB, int *INFO);
void zgtrtrs(const char *TRANS, const int *N, const int *NRHS, double
_Complex *DL, double _Complex *D, double _Complex *DU, double
_Complex *DU2, int *IPIV, double _Complex *B, const int *LDB, int *INFO);

```

参数说明:

参数名	类型	描述	输入/输出
trans	字符型	'N', $A \cdot X = B$ 'T', $A^{**T} \cdot X = B$ 'C', $A^{**T} \cdot X = B$ (共轭转置)	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数, $nrhs \geq 0$ 。	输入
dl	对应类型数组	维度 (n-1)。LU 分解中的 L 部分。	输入
d	对应类型数组	维度 (n)。U 的对角元素	输入
du	对应类型数组	为 U 的第一超对角元素。	输入
du2	对应类型数组	为 U 的第二超对角元素。	输入
ipiv	整型数组	转置数组。	输入
b	对应类型数组	右端项矩阵, 维度为 (ldb, nrhs)。	输入/输出
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入
info	整型数	执行结果。 = 0: 成功退出。 < 0: 如果 $INFO = -i$, 则第 i 个参数具有非法值。	输出

4.3.4 ?trtrs

接口说明: 求解具有多个右侧的三角形系数矩阵的线性方程组。

接口定义**Fotran 接口:**

```

subroutine ctrtrs (uplo, trans, diag, n, nrhs, a, lda, b, ldb, info);
subroutine dtrtrs (uplo, trans, diag, n, nrhs, a, lda, b, ldb, info);
subroutine strtrs (uplo, trans, diag, n, nrhs, a, lda, b, ldb, info);

```

subroutine ztrtrs (uplo, trans, diag, n, nrhs, a, lda, b, ldb, info);

C 接口:

void dtrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const double *a, const int *lda, double *b, const int *ldb, int *info);

void strtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const float *a, const int *lda, float *b, const int *ldb, int *info);

void ctrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const float _Complex *a, const int *lda, float _Complex *b, const int *ldb, int *info);

void ztrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const double _Complex *a, const int *lda, double _Complex *b, const int *ldb, int *info);

参数说明:

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
trans	字符型	'N', $A \cdot X = B$ 'T', $A^{**T} \cdot X = B$ 'C', $A^{**T} \cdot X = B$ (共轭转置)	输入
diag	字符型	'N', A 是非单位三角矩阵。 'U'. A 是单位三角矩阵。	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
nrhs	整型数	矩阵 B 的列数, $nrhs \geq 0$ 。	输入
a	对应类型数组	维度 (lda, n) 的三角矩阵 A。	输入
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
b	对应类型数组	右端项矩阵, 维度为 (ldb, nrhs)。	输入/输出
ldb	整型数	B 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入

参数名	类型	描述	输入/输出
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i, 则第 i 个参数具有非法值。 > 0: 如果 INFO = i, A 的第 i 个对角元素为零, 表明矩阵是奇异的, 并且解 X 尚未计算。	输出

4.4 矩阵求逆函数

4.4.1 ?getri

接口说明: 计算 LU 分解的一般矩阵的逆。

接口定义

Fortran 接口:

```
subroutine cgetri (n, a, lda, ipiv, work, lwork, info);
subroutine dgetri (n, a, lda, ipiv, work, lwork, info);
subroutine sgetri (n, a, lda, ipiv, work, lwork, info);
subroutine zgetri (n, a, lda, ipiv, work, lwork, info);
```

C 接口:

```
void sgetri_(const int *n, float *a, const int *lda, const int *ipiv, float *work,
const int *lwork, int *info);
void dgetri_(const int *n, double *a, const int *lda, const int *ipiv, double
*work, const int *lwork, int *info);
void cgetri_(const int *n, float _Complex *a, const int *lda, const int *ipiv,
float _Complex *work, const int *lwork, int *info);
void zgetri_(const int *n, double _Complex *a, const int *lda, const int *ipiv,
double _Complex *work, const int *lwork, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
a	对应类型数组	调用前: 矩阵 A 的 LU 分解结果 (由?getrf 得到)。 调用后: 保存原矩阵 A 的逆矩阵。	输出/输入
lda	整型数	a 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入

参数名	类型	描述	输入/输出
ipiv	整型数组	大小为 $\max(1, n)$ ，由 ?getrf 返回。	输入
work	对应类型数组	临时存储空间,使用 lwork=-1 调用后 work[0]为最优的 lwork 值。	输出
lwork	整型	work 数组的长度。lwork=-1 时查询最优 work 大小，结果保存在 work[0]中，否则要求 $\text{lwork} \geq n$ 。	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 $\text{INFO} = -i$ ，则第 i 个参数具有非法值 > 0: 如果 $\text{INFO} = i$ ，则 $U(i,i)$ 正好为零；该矩阵是奇异的，无法计算其逆矩阵。	输出

4.4.2 ?potri

接口说明：利用 Cholesky 分解计算对称（埃尔米特）正定矩阵的逆矩阵。

接口定义

Fortran 接口：

```
subroutine cpotri (uplo, n, a, lda, info);  
subroutine dpotri (uplo, n, a, lda, info);  
subroutine spotri (uplo, n, a, lda, info);  
subroutine zpotri (uplo, n, a, lda, info);
```

C 接口：

```
void spotri_(const char *UPLO, const int *N, const float *A, const int *LDA,  
int *INFO);  
void dpotri_(const char *UPLO, const int *N, const double *A, const int *LDA,  
int *INFO);  
void cpotri_(const char *UPLO, const int *N, const float _Complex *A, const  
int *LDA, int *INFO);  
void zpotri_(const char *UPLO, const int *N, const double _Complex *A,  
const int *LDA, int *INFO);
```

参数说明：

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
a	对应类型数组	调用前: 保存矩阵 A 的 Cholesky 分解结果(由?potrf 得到)。 调用后: 保存原矩阵 A 的逆矩阵。	输出/输入
lda	整型数	a 的 leading dimension 大小, 要求 $ldb \geq \max(1, n)$ 。	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i, 则第 i 个参数具有非法值。 > 0: 如果 INFO = i, 则因子 U 或 L 的 (i,i) 元素为零, 并且无法计算逆数。	输出

4.4.3 ?pptri

接口说明: 利用 Cholesky 分解计算压缩的对称 (埃尔米特) 正定矩阵的逆矩阵。

接口定义

Fotran 接口:

```
subroutine cpptri (uplo, n, ap, info);  
subroutine dpptri (uplo, n, ap, info);  
subroutine spptri (uplo, n, ap, info);  
subroutine zpptri (uplo, n, ap, info);
```

C 接口:

```
void spptri_(const char *UPLO, const int *N, const float *AP, int *INFO);  
void dpptri_(const char *UPLO, const int *N, const double *AP, int *INFO);  
void cpptri_(const char *UPLO, const int *N, const float _Complex *AP, int *INFO);  
void zpptri_(const char *UPLO, const int *N, const double _Complex *AP, int *INFO);
```

参数说明:

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入

参数名	类型	描述	输入/输出
n	整型数	矩阵 A 的维数， $n \geq 0$ 。	输入
ap	对应类型数组	调用前：如果 UPLO='U'时，以压缩格式保存矩阵 A 的 Cholesky 分解结果的 U 部分；如果 UPLO='L' 时，以压缩格式保存矩阵 A 的 Cholesky 分解结果的 L 部分。 调用后：保存原矩阵 A 的逆矩阵。	输出/输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i，则第 i 个参数具有非法值。 > 0: 如果 INFO = i，则因子 U 或 L 的 (i,i) 元素为零，并且无法计算逆数。	输出

4.4.4 ?trtri

接口说明：计算三角矩阵的逆。

接口定义

Fortran 接口：

```
subroutine ctrtri (uplo, diag, n, a, lda, info);
subroutine dtrtri (uplo, diag, n, a, lda, info);
subroutine strtri (uplo, diag, n, a, lda, info);
subroutine ztrtri (uplo, diag, n, a, lda, info);
```

C 接口：

```
void spptri_(const char *UPLO, const int *N, const float *AP, int *INFO);
void dpptri_(const char *UPLO, const int *N, const double *AP, int *INFO);
void cpptri_(const char *UPLO, const int *N, const float _Complex *AP, int *INFO);
void zpptri_(const char *UPLO, const int *N, const double _Complex *AP, int *INFO);
```

参数说明：

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
diag	字符型	'N': 矩阵 A 为非单位三角矩阵。 'U': 矩阵为单位三角矩阵。	输入/输出

参数名	类型	描述	输入/输出
n	整型数	矩阵 A 的维数， $n \geq 0$ 。	输入
a	对应类型数组	调用前：如果 UPLO='U'，则矩阵 A 包含了上三角部分，下三角部分不会使用；如果 UPLO='L'，则矩阵 A 包含了下三角部分，上三角部分不会使用；如果 DIAG='U'，则对角元素不会使用。 调用后：以相同格式存储矩阵 A 的逆矩阵。	输出/输入
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
info	整型数	执行结果。 = 0: 成功退出 < 0: 如果 INFO = -i，则第 i 个参数具有非法值。 > 0: 如果 INFO = i，A(i,i) 恰好为零。三角矩阵是奇异的，其逆矩阵无法计算。	输出

4.5 特征值问题函数

4.5.1 ?{he,sy}evd

接口说明：使用分而治之算法计算对称(Hermitian)矩阵的所有特征值和（可选）所有特征向量。

接口定义

Fortran 接口：

```
subroutine ssyevd (jobz, uplo, n, a, lda, w, work, lwork, iwork, liwork, info);
subroutine dsyevd (jobz, uplo, n, a, lda, w, work, lwork, iwork, liwork, info);
subroutine cheevd (jobz, uplo, n, a, lda, w, work, lwork, rwork, lrwork, iwork,
liwork, info);
subroutine zheevd (jobz, uplo, n, a, lda, w, work, lwork, rwork, lrwork, iwork,
liwork, info);
```

C 接口：

```
void dsyevd_(const char *jobz, const char *uplo, const int *n, double *a,
const int *lda, double *w, double *work, const int *lwork, int *iwork, const int
*liwork, int *info);
void ssyevd_(const char *jobz, const char *uplo, const int *n, float *a, const
int *lda, float *w, float *work, const int *lwork, int *iwork, const int *liwork, int
```

*info);

```
void cheevd_(const char *jobz, const char *uplo, const int *n, float
_Complex *a, const int *lda, float *w, float _Complex *work, const int *lwork,
float *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);
```

```
void zheevd_(const char *jobz, const char *uplo, const int *n, double
_Complex *a, const int *lda, double *w, double _Complex *work, const int
*lwork, double *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
jobz	字符型	'N': 只计算特征向量。 'V': 同时计算特征向量和特征值。	输入
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
a	对应类型数组	调用前: 保存待分解的对称矩阵。 调用后: 保存特征向量。	输出/输入
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
w	在 ssyevd/cheevd 中为单精度浮点型数组。 在 dsyevd/zheevd 中为双精度浮点型数组。	大小为 $\max(1, n)$, 包含按升序排列的矩阵 A 的特征值。	输出
work	对应类型数组	临时存储空间, 使用 $lwork = -1$ 调用后 $work[0]$ 为最优的 $lwork$ 值。	输出
lwork	整数型	work 数组的长度。 $lwork = -1$ 时查询最优 work 大小, 结果保存在 $work[0]$ 中, 否则: 1、当 $n \leq 1$ 时, $lwork \geq 1$ 。 2、当 $jobz = 'N'$ 且 $n > 1$ 时, $lwork \geq 2 * n + 1$ 。 3、当 $jobz = 'V'$ 且 $n > 1$ 时, $lwork \geq 1 + 6 * n + 2 * n * n$ 。	输入
rwork (复数独有)	在 cheevd 中为单精度实数。 在 zheevd 中为双精度实数。	临时存储空间, 调用后 $work[0]$ 为最优的 $lwork$ 值。	输出

参数名	类型	描述	输入/输出
lwork (复数独有)	在 cheevd 中为单精度复数型数组。 在 zheevd 中为双精度复数型数组。	rwork 数组的长度。 lwork=-1 时查询最优 work 大小, 结果保存在 work[0]中, 否则: 1、当 $n \leq 1$ 时, $lwork \geq 1$ 。 2、当 jobz='N'且 $n>1$ 时, $lwork \geq 2*n+1$ 。 3、当 jobz='V' 且 $n>1$ 时, $lwork \geq 1+6*n+2*n*n$ 。	输入
info	整型数	执行结果。 = 0: 成功。 < 0: 第 -info 个参数值不合法。 > 0: jobz='N'算法无法收敛, 无法计算第 info/(n+1)个特征值。	输出

4.5.2 ?{he,sy}ev

接口说明: 计算对称 (Hermite) 矩阵的所有特征值与特征向量。

接口定义

Fortran 接口:

```
subroutine ssyev(jobz, uplo, n, a, lda, w, work, lwork, info);
subroutine dsyev(jobz, uplo, n, a, lda, w, work, lwork, info);
subroutine cheev(jobz, uplo, n, a, lda, w, work, lwork, rwork, info);
subroutine zheev(jobz, uplo, n, a, lda, w, work, lwork, rwork, info);
```

C 接口:

```
void ssyev_(const char *jobz, const char *uplo, const int *n, float *A, const int *lda, float *w, float *work, const int *lwork, int *info);
void dsyev_(const char *jobz, const char *uplo, const int *n, double *A, const int *lda, double *w, double *work, const int *lwork, int *info);
void cheev_(const char *jobz, const char *uplo, const int *n, float _Complex *A, const int *lda, float *w, float _Complex *work, const int *lwork, float *rwork, int *info);
void zheev_(const char *jobz, const char *uplo, const int *n, double _Complex *A, const int *lda, double *w, double _Complex *work, const int *lwork, double *rwork, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
jobz	字符型	'N': 只计算特征向量。 'V': 同时计算特征向量和特征值。	输入
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
a	对应类型数组	调用前: 保存待分解的对称矩阵。 调用后: 保存特征向量。	输出/输入
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
w	在 ssyevd/cheevd 中为单精度浮点型数组。 在 dsyevd/zheevd 中为双精度浮点型数组。	大小为 $\max(1, n)$ 。如果 $info = 0$, 则包含按升序排列的矩阵 A 的特征值。	输出
work	对应类型数组	Info=0 时, work(0)返回最优 lwork 大小。	输出
lwork	整数型	Work 数组的长度。 lwork=-1 时查询最优 work 大小, 结果保存在 work[0]中。且 $lwork \geq \max(1, 2*n-1)$ 。	输入
rwork (复数独有)	在 ssyevd/cheevd 中为单精度浮点型数组。 在 dsyevd/zheevd 中为双精度浮点型数组。	工作数组, 取值 $\max(1, 3*n-1)$ 。	输出
info	整型数	执行结果。 = 0: 表示成功。 < 0: info=-i, 表示第 i 个参数非法。 > 0: 算法出错。	输出

4.5.3 ?steqr

接口说明: 计算简化为三对角形式的对称或埃尔米特矩阵的所有特征值和特征向量。

接口定义

Fortran 接口:

```
subroutine csteqr (compz, n, d, e, z, ldz, work, info);
```

```
subroutine dsteqr (compz, n, d, e, z, ldz, work, info);
```

```
subroutine ssteqr (compz, n, d, e, z, ldz, work, info);
```

```
subroutine zsteqr (compz, n, d, e, z, ldz, work, info);
```

C 接口:

```
void ssteqr_(const char *compz, const int *n, float *d, float *e, float *z, const  
int *ldz, float *work, int *info);
```

```
void dsteqr_(const char *compz, const int *n, double *d, double *e, double *z,  
const int *ldz, double *work, int *info);
```

```
void csteqr_(const char *compz, const int *n, float *d, float *e, float  
_Complex *z, const int *ldz, float *work, int *info);
```

```
void zsteqr_(const char *compz, const int *n, double *d, double *e, double  
_Complex *z, const int *ldz, double *work, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
compz	字符型	= 'N': 只计算特征值。 = 'V': 计算特征值和特征向量。 = 'I': 计算特征值和特征向量。	输入
n	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
d	对应类型数组	维度为 n。 调用前: 三对角矩阵的对角元素。 调用后: 若 info=0, 保存特征值, 并以升序排列。	输出/输入
e	对应类型数组	维度为 n-1。 调用前: 三对角矩阵的对角元素。 调用后: 销毁。	输出/输入
z	对应类型数组	维度为(ldz,n)。 调用前: 若 compz='V'时, 则为正交矩阵。 调用后: 1. 若 info=0, 且 compz='V'时, 则为原对称矩阵的正交特征向量。 2. 若 compz='I'时, 则为对称三对角矩阵的正交特征向量。 3. 若 compz='N'时, 不使用。	输出/输入

参数名	类型	描述	输入/输出
ldz	整数型	z 的 leading dimension 大小，要求 $ldz \geq \max(1, n)$ 。	输入
work	对应类型数组	工作数组。	输入
info	整数型	执行结果。 = 0: 成功退出 < 0: 如果 $INFO = -i$ ，则第 i 个参数具有非法值 > 0: 算法在总共 $30 \cdot N$ 次迭代中未能找到所有特征值；如果 $INFO = i$ ，则 E 的 i 个元素尚未收敛到零；退出时，D 和 E 包含与原始矩阵正交相似的对称三对角矩阵的元素。	输出

4.6 其他函数

4.6.1 ?{un,or}glq

接口说明：生成由?gelqf 形成的 LQ 分解的实/复矩阵 Q。

接口定义

Fortran 接口：

```
subroutine cunqlq (m, n, k, a, lda, tau, work, lwork, info);
subroutine dorglq (m, n, k, a, lda, tau, work, lwork, info);
subroutine sorglq (m, n, k, a, lda, tau, work, lwork, info);
subroutine zunglq (m, n, k, a, lda, tau, work, lwork, info);
```

C 接口：

```
void sorglq_(const int *m, const int *n, const int *k, float *a, const int *lda,
const float *tau, float *work, const int *lwork, int *info);
void dorglq_(const int *m, const int *n, const int *k, double *a, const int *lda,
const double *tau, double *work, const int *lwork, int *info);
void cunqlq_(const int *m, const int *n, const int *k, float _Complex *a, const
int *lda, const float _Complex *tau, float _Complex *work, const int *lwork,
int *info);
void zunglq_(const int *m, const int *n, const int *k, double _Complex *a,
const int *lda, const double _Complex *tau, double _Complex *work, const
```

```
int *lwork, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 Q 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 Q 的列数, $n \geq m$ 。	输入
k	整型数	基本反射器的数量, 其乘积定义矩阵 Q ($0 \leq k \leq m$)。	输入
a	对应类型数组	调用前: 第 i 行必须包含基本反射器 H(i) 向量, 其中 H(i) 由 ?gelqf 返回。 调用后: $m \times n$ 的矩阵 Q。	输出/输入
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	大小为 $\max(1, k)$, 是 ?gelqf 返回的数组。	输入
work	对应类型数组	Info=0 时, work(0) 返回最优 lwork 大小。	输出
lwork	整型数	数组 work 的维数。 $lwork \geq \max(1, m)$, 对于最优性能: $lwork \geq m \times nb$, 其中 nb 为最优块大小。如果 $lwork = -1$, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 $info = -i$, 则表示第 i 个参数为非法值。	输出

4.6.2 ?{un,or}gqr

接口说明: 生成由 ?geqrf 形成的具有正交行的实/复矩阵 Q。

接口定义**Fortran 接口:**

```
subroutine cungqr (m, n, k, a, lda, tau, work, lwork, info);
```

```
subroutine dorgqr (m, n, k, a, lda, tau, work, lwork, info);
```

```
subroutine sorgqr (m, n, k, a, lda, tau, work, lwork, info);
```

```
subroutine zungqr (m, n, k, a, lda, tau, work, lwork, info);
```

C 接口:

```
void sorgqr_(const int *m, const int *n, const int *k, float *a, const int *lda,
```

```

const float *tau, float *work, const int *lwork, int *info);
void dorgqr_(const int *m, const int *n, const int *k, double *a, const int *lda,
const double *tau, double *work, const int *lwork, int *info);
void cungqr_(const int *m, const int *n, const int *k, float _Complex *a, const
int *lda, const float _Complex *tau, float _Complex *work, const int *lwork,
int *info);
void zungqr_(const int *m, const int *n, const int *k, double _Complex *a,
const int *lda, const double _Complex *tau, double _Complex *work, const
int *lwork, int *info);

```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 Q 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 Q 的列数, $m \geq n$ 。	输入
k	整型数	基本反射器的数量, 其乘积定义矩阵 Q ($0 \leq k \leq n$)。	输入
a	对应类型数组	调用前: 第 i 列必须包含基本反射器 H(i) 向量, 其中 H(i) 由 ?geqrf 返回。 调用后: $m \times n$ 的矩阵 Q。	输出/输入
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	tau(i) 必须包含基本反射器 H(i) 的常量因子, 其由 ?gelqf 返回的。	输入
work	对应类型数组	Info=0 时, work(0) 返回最优 lwork 大小。	输出
lwork	整型数	数组 work 的维数。 $lwork \geq \max(1, n)$, 对于最优性能: $lwork \geq n \times nb$, 其中 nb 为最优块大小如果 $lwork = -1$, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 $info = -i$, 则表示第 i 个参数为非法值。	输出

4.6.3 ?{un,or}grq

接口说明: 生成由 ?gerqf 形成的具有正交行的实/复矩阵 Q。

接口定义

Fortran 接口:

```
subroutine cungrq (m, n, k, a, lda, tau, work, lwork, info);  
subroutine dorgrq (m, n, k, a, lda, tau, work, lwork, info);  
subroutine sorgrq (m, n, k, a, lda, tau, work, lwork, info);  
subroutine zungrq (m, n, k, a, lda, tau, work, lwork, info);
```

C 接口:

```
void sorgrq_(const int *M, const int *N, const int *K, float *A, const int *lda,  
float *tau, float *work, const int *lwork, int *info);  
void dorgrq_(const int *M, const int *N, const int *K, double *A, const int *lda,  
double *tau, double *work, const int *lwork, int *info);  
void cungrq_(const int *M, const int *N, const int *K, float _Complex *A,  
const int *lda, float _Complex *tau, float _Complex *work, const int *lwork,  
int *info);  
void zungrq_(const int *M, const int *N, const int *K, double _Complex *A,  
const int *lda, double _Complex *tau, double _Complex *work, const int  
*lwork, int *info);
```

参数说明:

参数名	类型	描述	输入/输出
m	整型数	矩阵 Q 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 Q 的列数, $n \geq 0$ 。	输入
k	整型数	基本反射器的数量, $(0 \leq k \leq m)$ 。	输入
a	对应类型数组	调用前: 第 i 列必须包含基本反射器 H(i) 向量, 其中 H(i) 由 ?gerqf 返回。 调用后: $m \times n$ 的矩阵 Q。	输出/输入
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	tau(i) 必须包含基本反射器 H(i) 的常量因子, 其由 ?gerqf 返回的。	输入
work	对应类型数组	Info=0 时, work(0) 返回最优 lwork 大小。	输出

参数名	类型	描述	输入/输出
lwork	整型数	数组 work 的维数。lwork ≥ max(1, M)，对于最优性能：lwork ≥ M*Nb，其中 Nb 为最优块大小如果 lwork=-1，则该例程只计算 work 数组的最优大小，并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 info=-i，则表示第 i 个参数为非法值。	输出

4.6.4 ?{or,un}gql

接口说明：生成由?geqlf 形成的具有正交行的实/复矩阵 Q。

接口定义

Fotran 接口：

```
subroutine cungql (m, n, k, a, lda, tau, work, lwork, info)
subroutine dorgql (m, n, k, a, lda, tau, work, lwork, info)
subroutine sorgql (m, n, k, a, lda, tau, work, lwork, info)
subroutine zungql (m, n, k, a, lda, tau, work, lwork, info)
```

C 接口：

```
void sorgql_(const int *m, const int *n, const int *k, float *a, const int *lda,
float *tau, float *work, const int *lwork, int *info);
void dorgql_(const int *m, const int *n, const int *k, double *a, const int *lda,
double *tau, double *work, const int *lwork, int *info);
void cungql_(const int *m, const int *n, const int *k, float _Complex *a, const
int *lda, float _Complex *tau, float _Complex *work, const int *lwork, int
*info);
void zungql_(const int *m, const int *n, const int *k, double _Complex *a,
const int *lda, double _Complex *tau, double _Complex *work, const int
*lwork, int *info);
```

参数说明：

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
m	整型数	矩阵 Q 的行数， $m \geq 0$ 。	输入
n	整型数	矩阵 Q 的列数， $m \geq n \geq 0$ 。	输入
k	整型数	基本反射器的数量， $(0 \leq k \leq n)$ 。	输入
a	对应类型数组	调用前：第 i 列必须包含基本反射器 H(i) 向量，其中 H(i) 由 ?geqlf 返回。 调用后：m*n 的矩阵 Q。	输出/输入
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	tau(i) 必须包含基本反射器 H(i) 的常量因子，其由 ?geqlf 返回的。	输入
work	对应类型数组	Info=0 时，work(0) 返回最优 lwork 大小。	输出
lwork	整型数	数组 work 的维数。 $lwork \geq \max(1, M)$ ，对于最优性能： $lwork \geq N \cdot NB$ ，其中 NB 为最优块大小。如果 $lwork = -1$ ，则该例程只计算 work 数组的最优大小，并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0：成功退出。 info<0：若 $info = -i$ ，则表示第 i 个参数为非法值。	输出

4.6.5 ?{or,un}mlq

接口说明：利用 ?geqlf 得到的 Q 计算 $C = Q^* \times C$ 或者 $C = C \times Q^*$ 。当 Q 转置，实数时 Q^* 表示 Q^T ，复数时 Q^* 表示 Q^H ；当 Q 不转置， Q^* 表示 Q。

接口定义

Fortran 接口：

```
subroutine cunmlq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine dormlq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine sormlq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine zunmlq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
```

C 接口：

```
void sormlq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float *a, const int *lda, const float *tau, float *c, const int *ldc,
float *work, const int *lwork, int *info);
```

```
void dormlq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double *a, const int *lda, const double *tau, double *c, const int
*ldc, double *work, const int *lwork, int *info);
```

```
void cunmlq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float _Complex *a, const int *lda, const float _Complex *tau,
float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int
*info);
```

```
void zunmlq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double _Complex *a, const int *lda, const double _Complex *tau,
double _Complex *c, const int *ldc, double _Complex *work, const int *lwork,
int *info);
```

参数说明:

参数名	类型	描述	输入/输出
side	字符型	"L": Q^* 在左边。 "R": Q^* 在右边。	输入
trans	字符型	"N": Q 不转置。 "T": Q 转置。	输入
m	整型数	矩阵 c 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 c 的列数, $n \geq 0$ 。	输入
k	整型数	生成 Q 的元素反射器的个数。 side="L" 时, $m \geq k \geq 0$ 。 side="R" 时, $n \geq k \geq 0$ 。	输入
a	对应类型数组	当 side="L" 时, 维数为 (lda, m)。 当 side="R" 时, 维数为 (lda, n)。	输入
lda	整型数	a 的 leading dimension 大小, 要求 $lda \geq \max(1, k)$ 。	输入
tau	对应类型数组	tau(i) 必须包含基本反射器 $H(i)$ 的常量因子, 其 由 ?gelqf 返回的。	输入
c	对应类型数组	待计算矩阵, 维数为 (ldc, n)。	输入/输出
ldc	整型数	c 的 leading dimension 大小, 要求 $ldc \geq \max(1, m)$ 。	输入
work	对应类型数组	维数 ($\max(1, lwork)$)。Info=0 时, work(0) 返 回最优 lwork 大小。	输出

参数名	类型	描述	输入/输出
lwork	整数型	当 side="L" 时, $lwork \geq \max(1, n)$ 。 当 side="R" 时, $lwork \geq \max(1, m)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 info=-i, 则表示第 i 个参数为非法值。	输出

4.6.6 ?{or,un}mqr

接口说明: 利用?geqrf 得到的 Q 计算 $C = Q^* \times C$ 或者 $C = C \times Q^*$ 。当 Q 转置, 实数时 Q^* 表示 Q^T , 复数时 Q^* 表示 Q^H ; 当 Q 不转置, Q^* 表示 Q。

接口定义

Fortran 接口:

```
subroutine cunmqr(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine dormqr(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine sormqr(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine zunmqr(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
```

C 接口:

```
void sormqr_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float *a, const int *lda, float *tau, float *c, const int *ldc, float
*work, const int *lwork, int *info);
void dormqr_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double *a, const int *lda, double *tau, double *c, const int *ldc,
double *work, const int *lwork, int *info);
void cunmqr_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float _Complex *a, const int *lda, float _Complex *tau, float
_Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);
void zunmqr_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double _Complex *a, const int *lda, double _Complex *tau,
double _Complex *c, const int *ldc, double _Complex *work, const int *lwork,
int *info);
```

参数说明:

参数名	类型	描述	输入/输出
side	字符型	"L": Q^* 在左边。 "R": Q^* 在右边。	输入
trans	字符型	"N": Q 不转置。 "T": Q 转置。	输入
m	整型数	矩阵 c 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 c 的列数, $n \geq 0$ 。	输入
k	整型数	生成 Q 的元素反射器的个数。 side="L"时, $m \geq k \geq 0$ 。 side="R"时, $n \geq k \geq 0$ 。	输入
a	对应类型数组	原矩阵	输入
lda	整型数	矩阵 A 的主维。 当 side="L"时, $lda \geq \max(1, m)$ 。 当 side="R"时, $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	tau(i)必须包含基本反射器 $H(i)$ 的常量因子, 其由?geqrf 返回的。	输入
c	对应类型数组	待计算矩阵, 维数为 (ldc, n) 。	输入/输出
ldc	整型数	c 的 leading dimension 大小, 要求 $ldc \geq \max(1, m)$ 。	输入
work	对应类型数组	维数 $(\max(1, lwork))$ 。Info=0 时, work(0)返回最优 lwork 大小。	输出
lwork	整型数	当 side="L"时, $lwork \geq \max(1, n)$ 。 当 side="R"时, $lwork \geq \max(1, m)$ 。 如果 $lwork = -1$, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 $info = -i$, 则表示第 i 个参数为非法值。	输出

4.6.7 ?{or,un}mql

接口说明: 利用?geqlf 得到的 Q 计算 $C = Q^* \times C$ 或者 $C = C \times Q^*$ 。当 Q 转置, 实数时 Q^* 表示 Q^T , 复数时 Q^* 表示 Q^H ; 当 Q 不转置, Q^* 表示 Q 。

接口定义

Fotran 接口:

```
subroutine cunmql(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine dormql(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine sormql(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine zunmql(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
```

C 接口:

```
void sormql_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float *a, const int *lda, const float *tau, float *c, const int *ldc,
float *work, const int *lwork, int *info);
```

```
void dormql_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double *a, const int *lda, const double *tau, double *c, const int
*ldc, double *work, const int *lwork, int *info);
```

```
void cunmql_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float _Complex *a, const int *lda, const float _Complex *tau,
float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int
*info);
```

```
void zunmql_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double _Complex *a, const int *lda, const double _Complex *tau,
double _Complex *c, const int *ldc, double _Complex *work, const int *lwork,
int *info);
```

参数说明:

参数名	类型	描述	输入/输出
side	字符型	"L": Q^* 在左边。 "R": Q^* 在右边。	输入
trans	字符型	"N": Q 不转置。 "T": Q 转置。	输入
m	整型数	矩阵 c 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 c 的列数, $n \geq 0$ 。	输入
k	整型数	生成 Q 的元素反射器的个数。 side="L"时, $m \geq k \geq 0$ 。 side="R"时, $n \geq k \geq 0$ 。	输入
a	对应类型数组	原矩阵, 维数 (lda, k)。	输入

参数名	类型	描述	输入/输出
lda	整型数	矩阵 A 的主维。 当 side="L" 时, $lda \geq \max(1, m)$ 。 当 side="R" 时, $lda \geq \max(1, n)$ 。	输入
tau	对应类型数组	tau(i) 必须包含基本反射器 H(i) 的常量因子, 其由 ?geqlf 返回的。	输入
c	对应类型数组	待计算矩阵, 维数为 (ldc, n)。	输入/输出
ldc	整型数	c 的 leading dimension 大小, 要求 $ldc \geq \max(1, m)$ 。	输入
work	对应类型数组	维数(max(1,lwork))。Info=0 时, work(0) 返回最优 lwork 大小。	输出
lwork	整型数	当 side="L" 时, $lwork \geq \max(1, n)$ 。 当 side="R" 时, $lwork \geq \max(1, m)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 info=-i, 则表示第 i 个参数为非法值。	输出

4.6.8 ?{or,un}mrq

接口说明: 利用 ?gerqf 得到的 Q 计算 $C = Q^* \times C$ 或者 $C = C \times Q^*$ 。当 Q 转置, 实数时 Q^* 表示 Q^T , 复数时 Q^* 表示 Q^H ; 当 Q 不转置, Q^* 表示 Q。

接口定义

Fotran 接口:

```
subroutine cunmrq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine dormrq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine sormrq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
subroutine zunmrq(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info);
```

C 接口:

```
void sormrq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float *a, const int *lda, float *tau, float *c, const int *ldc, float
*work, const int *lwork, int *info);
void dormrq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double *a, const int *lda, double *tau, double *c, const int *ldc,
```



```
double *work, const int *lwork, int *info);
void cunmrq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, float _Complex *a, const int *lda, float _Complex *tau, float
_Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);
void zunmrq_(const char *side, const char *trans, const int *m, const int *n,
const int *k, double _Complex *a, const int *lda, double _Complex *tau,
double _Complex *c, const int *ldc, double _Complex *work, const int *lwork,
int *info);
```

参数说明:

参数名	类型	描述	输入/输出
side	字符型	"L": Q^* 在左边。 "R": Q^* 在右边。	输入
trans	字符型	"N": Q 不转置。 "T": Q 转置。	输入
m	整型数	矩阵 c 的行数, $m \geq 0$ 。	输入
n	整型数	矩阵 c 的列数, $n \geq 0$ 。	输入
k	整型数	生成 Q 的元素反射器的个数。 side="L"时, $m \geq k \geq 0$ 。 side="R"时, $n \geq k \geq 0$ 。	输入
a	对应类型数组	当 side="L"时, 维数为 (lda, m)。 当 side="R"时, 维数为 (lda, n)。	输入
lda	整型数	矩阵 A 的主维, $lda \geq \max(1, K)$ 。	输入
tau	对应类型数组	tau(i) 必须包含基本反射器 $H(i)$ 的常量因子, 其由 ?gerqf 返回的。	输入
c	对应类型数组	待计算矩阵, 维数为 (ldc, n)。	输入/输出
ldc	整型数	c 的 leading dimension 大小, 要求 $ldc \geq \max(1, m)$ 。	输入
work	对应类型数组	维数(max(1,lwork))。Info=0 时, work(0)返回最优 lwork 大小。	输出
lwork	整型数	当 side="L"时, $lwork \geq \max(1, n)$ 。 当 side="R"时, $lwork \geq \max(1, m)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入

参数名	类型	描述	输入/输出
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 info=-i, 则表示第 i 个参数为非法值。	输出

4.6.9 ?{he,sy}trd

接口说明：将对称矩阵或 Hermite 矩阵通过相似变换成对称三对角 T。

接口定义

Fotran 接口：

```
subroutine chetrd (uplo, n, a, lda, d, e, tau, work, lwork, info);
subroutine dsytrd (uplo, n, a, lda, d, e, tau, work, lwork, info);
subroutine ssytrd (uplo, n, a, lda, d, e, tau, work, lwork, info);
subroutine zhetrd (uplo, n, a, lda, d, e, tau, work, lwork, info);
```

C 接口：

```
void ssytrd_(const char *uplo, const int *n, float *a, const int *lda, float *d,
float *e, float *tau, float *work, const int *lwork, int *info);
void dsytrd_(const char *uplo, const int *n, double *a, const int *lda, double
*d, double *e, double *tau, double *work, const int *lwork, int *info);
void chetrd_(const char *uplo, const int *n, float _Complex *a, const int *lda,
float *d, float *e, float _Complex *tau, float _Complex *work, const int *lwork,
int *info);
void zhetrd_(const char *uplo, const int *n, double _Complex *a, const int
*lda, double *d, double *e, double _Complex *tau, double _Complex *work,
const int *lwork, int *info);
```

参数说明：

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。 'L': A 中保存下三角矩阵。	输入
n	整型数	矩阵 A 的维数， $n \geq 0$ 。	输入
a	对应类型数组	调用前：保存待分解的对称矩阵。 调用后：保存特征向量。	输出/输入

参数名	类型	描述	输入/输出
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
d	在 ssytrd/chetrd 中为单精度浮点型数组。 在 dsytrd/zhetrd 中为双精度浮点型数组。	三对角矩阵 T 的对角元素，即 $d(i)=a(i,i)$ 。	输出
e	在 ssytrd/chetrd 中为单精度浮点型数组。 在 dsytrd/zhetrd 中为双精度浮点型数组。	三对角矩阵 T 非对角元素。 当 UPLO="U"时， $e(i)=a(i,i+1)$ 。 当 UPLO="L"时， $e(i)=a(i+1,i)$ 。	输出
tau	对应类型数组	$\tau(i)$ 必须包含基本反射器的常量因子，维数为 $n-1$ 。	输出
work	对应类型数组	维数($\max(1,lwork)$)。Info=0 时，work(0)返回最优 lwork 大小。	输出
lwork	整型数	work 数组的长度。 $lwork=-1$ 时查询最优 work 大小，结果保存在第一个元素当中。	输入
info	整型数	执行结果。 info=0: 成功退出。 info<0: 若 $info=-i$ ，则表示第 i 个参数为非法值。	输出

4.6.10 ?lasr

接口说明：将一系列平面旋转应用于一般的矩形矩阵。

接口定义

Fortran 接口：

```
subroutine clasr (side, pivot, direct, m, n, c, s, a, lda);
subroutine dlasr (side, pivot, direct, m, n, c, s, a, lda);
subroutine slasr (side, pivot, direct, m, n, c, s, a, lda);
```

```
subroutine zlasr (side, pivot, direct, m, n, c, s, a, lda);
```

C 接口:

```
void slasr_(const char *SIDE, const char *PIVOT, const char *DIRECT,  
const int *M, const int *N, float *C, float *S, float *A, const int *LDA);
```

```
void dlasr_(const char *SIDE, const char *PIVOT, const char *DIRECT,  
const int *M, const int *N, double *C, double *S, double *A, const int *LDA);
```

```
void clasr_(const char *SIDE, const char *PIVOT, const char *DIRECT,  
const int *M, const int *N, float *C, float *S, float _Complex *A, const int  
*LDA);
```

```
void zlasr_(const char *SIDE, const char *PIVOT, const char *DIRECT,  
const int *M, const int *N, double *C, double *S, double _Complex *A, const  
int *LDA);
```

参数说明:

参数名	类型	描述	输入/输出
side	字符型	'L', $A:=P*A$ 。 'R', $A:=A*P^T$ 。	输入
pivot	字符型	= 'V': 可变枢轴, 平面 $(k,k+1)$ 。 = 'T': 顶部枢轴, 平面 $(1,k+1)$ 。 = 'B': 底部枢轴, 平面 (k,z) 。	输入
direct	字符型	= 'F': 向前, $P = P(z-1)*...*P(2)*P(1)$ = 'B': 向后, $P = P(1)*P(2)*...*P(z-1)$	输入
m	整型数	矩阵 a 的行数。	
n	整型数	矩阵 a 的列数。	输入
c	在 slasr/clasr 中为单精度浮点型数组。 在 dlasr/zlasr 中为双精度浮点型数组。	平面旋转的余弦 $c(k)$ 。 side="L"时, 维度为 $(m-1)$ 。 side="R"时, 维度为 $(n-1)$ 。	输入
s	在 slasr/clasr 中为单精度浮点型数组。 在 dlasr/zlasr 中为双精度浮点型数组。	平面旋转的正弦 $s(k)$ 。 side="L"时, 维度为 $(m-1)$ 。 side="R"时, 维度为 $(n-1)$ 。	输入
a	对应类型数组	维度为 (lda,n)	输入/输出
lda	整型数	a 的 leading dimension 大小, 要求 $lda \geq \max(1,m)$ 。	输入

4.6.11 ?{he,sy}trd

接口说明： 将对称矩阵或 Hermite 矩阵 A 转为对称或 Hermite 三对角矩阵 T。

接口定义

Fortran 接口：

```
subroutine chetrd_2stage (vect, uplo, n, a, lda, d, e, tau, hous2, lhous2,  
work, lwork, info);
```

```
subroutine dsytrd_2stage (vect, uplo, n, a, lda, d, e, tau, hous2, lhous2,  
work, lwork, info);
```

```
subroutine ssytrd_2stage (vect, uplo, n, a, lda, d, e, tau, hous2, lhous2,  
work, lwork, info);
```

```
subroutine zhetrd_2stage (vect, uplo, n, a, lda, d, e, tau, hous2, lhous2,  
work, lwork, info);
```

C 接口：

```
void ssytrd_2stage_(const char *VECT, const char *UPLO, const int *N,  
float *A, const int *LDA, float *D, float *E, float *TAU, float *HOUS2, int  
*LHOUS2, float *WORK, const int *LWORK, int *INFO);
```

```
void dsytrd_2stage_(const char *VECT, const char *UPLO, const int *N,  
double *A, const int *LDA, double *D, double *E, double *TAU, double  
*HOUS2, int *LHOUS2, double *WORK, const int *LWORK, int *INFO);
```

```
void chetrd_2stage_(const char *VECT, const char *UPLO, const int *N,  
float _Complex *A, const int *LDA, float *D, float *E, float _Complex *TAU,  
float _Complex *HOUS2, int *LHOUS2, float _Complex *WORK, const int  
*LWORK, int *INFO);
```

```
void zhetrd_2stage_(const char *VECT, const char *UPLO, const int *N,  
double _Complex *A, const int *LDA, double *D, double *E, double  
_Complex *TAU, double _Complex *HOUS2, int *LHOUS2, double  
_Complex *WORK, const int *LWORK, int *INFO);
```

参数说明：

参数名	类型	描述	输入/输出
vect	字符型	'N': 不需要 Householder 表达式。 'V': 需要 Householder 表达式。	输入

参数名	类型	描述	输入/输出
uplo	字符型	'U': 存放 A 的上三角部分。 'L': 存放 A 的下三角部分。	输入
n	整型数	矩阵 A 的阶数。 $N \geq 0$ 。	输入
a	对应类型数组	维度为(la,n)。	输入/输出
lda	整型数	a 的 leading dimension 大小， 要求 $lda \geq \max(1,n)$ 。	输入
d	在 ssytrd_2stage/chetrd_2stage 中为单精度浮点型数组。 在 dsytrd_2stage/zhetrd_2stage 中为双精度浮点型数组。	维度为 n，对称三对角矩阵 T 的 对角元素。	输出
e	在 ssytrd_2stage/chetrd_2stage 中为单精度浮点型数组。 在 dsytrd_2stage/zhetrd_2stage 中为双精度浮点型数组。	维度为 n-1，对称三对角矩阵 T 的非对角元素。	输出
tau	对应类型数组。	第一级基本反射面的标量因子	输出
hous2	对应类型数组。	存放第二阶段中 Householder 表达形式。	输出
lhous2	整型数	= 'N', $lhous2 = \max(1, 4*n)$; = 'V', 则选项尚不可用。	输入
work	对应类型数组	工作数组，维度为 lwork。	输出
lwork	整数型	工作数组的长度。	输入
info	整型数	执行结果。 =0: 成功退出。 <0: 如果 info=i, 则第 i 个参数 具有非法值。	输出